Errand Agent: design note

Rule: the model suggests, plain code decides. The model reads requests, picks tools and writes replies. Code owns approval, idempotency, retries, limits, memory rules and context.

State model

Each task is a row in SQLite with a state from a transition table: `COLLECTING → SEARCHING → COMPARING → HOLDING → AWAITING_APPROVAL → EXECUTING → VERIFYING → DONE`, plus `NEEDS_USER` and `FAILED`. Kinds are book, reschedule, cancel and rebook. An illegal move throws and is traced. The model sends partial `update_request` calls. Code merges them, so a correction changes one field, works out what is missing, and refuses a question about a known field (`reask_blocked`). `hold_slot` only accepts an option code offered, after the user has spoken.

Approval enforcement

`request_approval` takes no arguments. Code builds the card from stored records, using the held price. Approval can only come from the user's raw message while a task is `AWAITING_APPROVAL` (a short, unconditional yes) or the Approve button with that approval ID. The model cannot grant it, and tool output never reaches the check. Each approval stores a sha256 fingerprint of user, action, provider, slot, price and hold or booking ID. Just before acting, code rebuilds it from current records. Any change or an expired hold voids it: hold again, ask again. Approvals are single use.

Idempotency and recovery

The key is built from the action and approval ID, and the model never sees it. One transaction consumes the approval and writes a `pending_ops` row before any tool runs. On a timeout, code polls `get_booking_status` with bounded backoff. Booked is success. Not booked retries with the same key if the hold is valid, otherwise it needs a new hold and approval. Unknown is reported as unconfirmed, with no retry, and the op stays open. Code verifies status before saying "booked", even after a plain success. At startup, every open op is settled through the source of truth. A test SIGKILLs a real child process at both crash points. The mock has its own SQLite file, so nothing can cheat with a shared transaction.

Memory and context

Facts are explicit (from the user) or inferred (from booking history, unconfirmed until the user confirms). Every write must quote the user's exact words from this session, so a review cannot plant a fact. Correcting supersedes a row. Forgetting blanks it, leaves a tombstone, scrubs chat, summary, queued messages and the active task, and ends the run so the value cannot return in that loop. `buildContext` is deterministic with a 3,000-token budget: only active, explicit or confirmed, relevant facts, ranked and trimmed, plus a code-built summary and recent turns.

Observability

One JSONL file per run: state transitions, model calls with latency, tokens and cost, tool calls with sanitized results, gate decisions, approvals, limits, and a `context_manifest` each turn listing memory IDs in, IDs out with reasons, token counts and a payload hash. The mock logs side effects to `mock_calls`. Eval "actual" values come from these two sources, and 7 invariants run after every scenario.

Key trade-offs

- **Single agent.** One user and one task at a time. Safety comes from code, not from a second model checking the first, and more agents would add hand-offs at the points that need the most control.
- **An executor runs actions, not the model.** Recovery never depends on the model, at the cost of code-written outcome messages. The model-facing `book`, `cancel` and `reschedule` tools still exist as tripwires: a model that acts early, for example after an injection, is blocked and traced.
- **Capability over detection.** Nothing in a tool result can grant approval, change the request or write memory. The injection detector only flags text in the trace.
- **Append-only tool loop.** Thinking blocks must not be edited, so a run that outgrows the budget ends with a clear message rather than trimming history.
- **One extra turn.** Search, pick, hold, then approve costs a turn, but approval always covers the real held price.

Annotated trace: timeout after success

```
PENDING: replaced with an excerpt from the real claude-sonnet-5-5 run in docs/traces/timeout-after-success.jsonl
approval_granted  via=chat                          <- code read the raw "yes"
approval_consumed op written before any tool        <- crash-safe from here
AWAITING_APPROVAL -> EXECUTING
tool_call book attempt=1 -> error timeout           <- the booking really happened, the response was lost
EXECUTING -> VERIFYING (book_timeout)               <- never guess
tool_call get_booking_status -> booked B-0003       <- source of truth, so no retry
gate_decision verified                              <- only now may we say booked
VERIFYING -> DONE, run_end action_result
```